深度学习基础

第 6 讲 — 卷积神经网络基础

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与本讲目标
2. 从全连接到卷积：两个关键先验
3. 卷积运算详解
4. 多通道卷积
5. 池化层
6. 感受野
7. LeNet：CNN 的鼻祖
8. 小结与下讲预告

第 1 讲 导论 → 第 2 讲 数学基础 → 第 3 讲 MLP+BP → 第 4 讲 训练技巧 → 第 5 讲 PyTorch

你已经掌握了通用神经网络的 “通用工具箱”

- **建模**：MLP + 反向传播 + ReLU
- **训练**：He 初始化 + BatchNorm + Dropout + Adam + 学习率调度
- **编程**：PyTorch + Dataset + 完整训练循环

下一个问题

这些工具是**通用的**。但通用 $\neq$ 最优。当数据有特殊结构时，通用模型会失效。**图像就是这样的数据。**

用 MLP 分类一张 224×224 的 RGB 图片

- 输入维度：$224 \times 224 \times 3 = 150528$
- 假设第一个隐藏层有 1000 个神经元
- 第一层参数量：$150528 \times 1000 \approx 1.5$ 亿
- 这只是**一层**！整个网络可能上十亿参数

三个致命问题

1. 参数过多 $\rightarrow$ 容易过拟合
2. 计算量爆炸 $\rightarrow$ 训练极慢
3. **忽略了图像的空间结构**

对比：LeNet 处理 28×28 MNIST

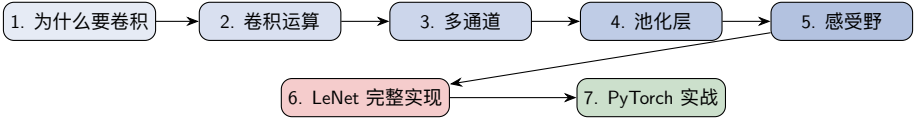
- MLP 参数: $\sim$ 100 万
- LeNet 参数: $\sim$ 6 万
- **相差近 20 倍**
- 而 LeNet 精度更高

核心矛盾

图像有强烈的**先验结构**，全连接层完全没用上这一点。

一句话目标

理解**卷积**与**池化**两种新算子，并用它们搭建第一个真正的视觉模型——**LeNet**。



学完本讲，你将能

- 解释为什么 CNN 比 MLP 更适合图像
- 手动计算卷积输出尺寸与参数量
- 用 PyTorch 从零搭建 LeNet 并训练 MNIST

观察

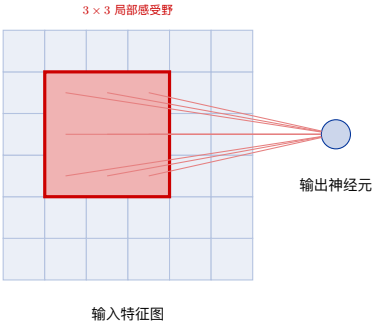
图像中相邻像素**高度相关**，远处像素几乎独立。识别一个“眼睛”只需看眼睛附近的像素，不需要看图像角落。

推论：稀疏连接

每个神经元只需要连接**局部区域**的输入，而不是所有像素。这种局部区域叫**感受野**。

参数对比（看一个 5×5 区域）

- 全连接：每个输出连接所有像素 $\rightarrow$ 大量冗余
- 局部连接：每个输出只连接 $5 \times 5 = 25$ 个输入



观察

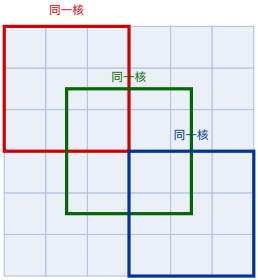
“猫”出现在图像左上角和右下角，**都是同一只猫**。识别“猫”的特征检测器，应该在任何位置都能用。

推论：参数共享

同一个特征检测器（比如“边缘检测器”）的参数，应该在所有位置**共享**。

这正是卷积的定义

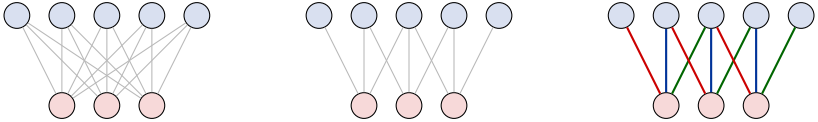
卷积 = 局部连接 + 参数共享
一个 $k \times k$ 的“小窗口”（卷积核），**滑过整张图**，在每个位置做相同的运算。



参数完全共享

设定：5 个输入 → 3 个输出（一维）

关键约定：每条边的**颜色 = 参数的 “身份”**。**颜色相同 = 同一个 w（共享）**；颜色不同 = 各自独立的 w。



全连接

每输出连 **5** 边
共 15 条边
15 个独立 w

局部连接 ($k=3$)

每输出连 **3** 边
共 9 条边
9 个独立 w

卷积 ($k=3$ 共享)

每输出连 **3** 边
9 边 **只 3 色**
只 3 个独立 w

看图记住两个定义

每输出连接数：一个输出节点向上 “看” 几条边。**独立参数**：整张图里互不相同的 w 的个数。图 3 同色 = 同一个 w，9 条边只有 3 个独立 w ——这就是**参数共享**。

方案	每输出连接数	是否参数共享	总独立 w 数 (输入 32^2, 输出 32^2)
全连接	1024	否	$1024 \times 1024 \approx$ **100 万**
局部连接（5×5）	25	否	$1024 \times 25 \approx$ **2.5 万**
局部 + 共享（卷积）	25	**是（共享!）**	**25**

差距：4 个数量级

全连接 → 局部连接：参数 $\div 40$（由“局部性”先验贡献）；局部连接 → 卷积：参数再 $\div 1000$（由“参数共享”贡献）。**这就是 CNN 能训练得动的原因。**

重要观点：归纳偏置（Inductive Bias）

卷积把先验知识“硬编码”进网络结构。**好的先验 = 更少的数据 + 更好的泛化。**

数学定义

输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$ 与卷积核 $\mathbf{K} \in \mathbb{R}^{k_h \times k_w}$：

$$Y_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} K_{a,b} \cdot X_{i+a,\, j+b}$$

白话翻译

(1) 把卷积核**对齐**到输入左上角；(2) **逐元素相乘再求和**得输出一个数；(3) 卷积核向右滑动一格重复；一行扫完后下移一格继续。

数学小注

严格的 “卷积” 要先翻转核，CNN 中我们不翻转，本质是**互相关**；因为参数是学出来的，业界统一叫 “卷积”。

输入 X 与卷积核 K

$$\mathbf{X} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$$

第一个输出元素

$Y_{0,0} = 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 5 = \mathbf{25}$

滑动后……

$Y_{0,1} = 0 \cdot 2 + 1 \cdot 3 + 2 \cdot 5 + 3 \cdot 6 = 31$

$Y_{1,0} = 0 \cdot 4 + 1 \cdot 5 + 2 \cdot 7 + 3 \cdot 8 = 43$

$Y_{1,1} = 0 \cdot 5 + 1 \cdot 6 + 2 \cdot 8 + 3 \cdot 9 = 49$

2×2 核位置 1

1	2	3
4	5	6
7	8	9

输入 **X**

25	31
43	49

输出 **Y**

观察

输入 3×3 + 核 2×2 = 输出 $\mathbf{2 \times 2}$

无填充情况下

输入 $n \times n$，卷积核 $k \times k \Rightarrow$ 输出尺寸 $\text{out} = n - k + 1$

验证

$n = 3, k = 2 \Rightarrow 2$ ； $n = 28, k = 5 \Rightarrow 24$（MNIST 第一层）

两个问题

(1) 输出不断变小：经过多层卷积后特征图缩到很小，限制网络深度；
(2) 边角像素被使用次数远少于中心像素：信息丢失不均匀。

解决方法

填充（Padding）：在输入周围补一圈 0

Same Padding

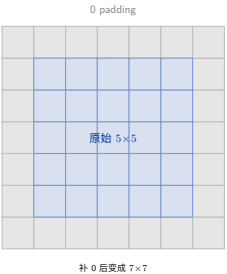
在输入四周各补 p 行/列的 0，让输出和输入**尺寸相同**。

新输出公式

$\text{out} = n + 2p - k + 1$。要让输出 = 输入：
$p = (k-1)/2$

常见做法

卷积核都用**奇数大小**（3、5、7），这样 p 是整数。
$\mathbf{3 \times 3 + p = 1}$ 是最常见组合。



什么是 Stride

卷积核每次滑动**多少格**。默认 stride=1，stride=2 表示每次滑 2 格，输出尺寸减半。

完整输出公式（很重要！）

$\boxed{\text{out} = \lfloor (n + 2p - k)/s \rfloor + 1}$ （n 边长，p 填充，k 核，s 步幅）

练习：套公式（三个常见配置）

- 输入 32×32，$k = 3$，$p = 1$，$s = 1 \Rightarrow$ **32**（尺寸不变）
- 输入 32×32，$k = 3$，$p = 1$，$s = 2 \Rightarrow$ **16**（尺寸减半）
- 输入 224×224，$k = 7$，$p = 3$，$s = 2 \Rightarrow$ **112**（ResNet 首层）

RGB 图像 = 3 个通道

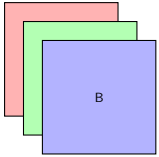
一张 224×224 彩色图的真实形状是 $\mathbf{3 \times 224 \times 224}$。三个通道分别记录红/绿/蓝的强度。

PyTorch 约定：(N, C, H, W)

N = Batch, C = Channels, H = Height, W = Width

关键问题

卷积核如何处理多通道？**答案：卷积核也变成 3D。**



3 通道堆叠 $3 \times H \times W$

规则

输入有 C_{in} 个通道，卷积核也必须有 C_{in} 个通道。每个通道独立做 2D 卷积，然后**把所有通道结果相加**得到一个 2D 输出。

形状演算

输入 $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$，卷积核 $\mathbf{K} \in \mathbb{R}^{C_{\text{in}} \times k \times k}$

$$Y_{i,j} = \sum_{c} \sum_{a,b} K_{c,a,b} \cdot X_{c,i+a,j+b}$$

输出是**单通道**的 2D 特征图：$\mathbf{Y} \in \mathbb{R}^{H' \times W'}$

参数量

一个 3×3 卷积核处理 RGB 图：$3 \times 3 \times 3 = 27$ 个参数（$+1$ 偏置 $= 28$）

为什么需要多个输出通道?

一个卷积核只能提取**一种特征**（如水平边缘）。我们希望同时提取多种特征：水平/垂直边缘、纹理、颜色对比……

做法

用 C_{out} 个不同的卷积核，每个产生一张特征图，最终输出 C_{out} 个通道。

标准卷积层的形状

输入: (C_{in}, H, W)
权重: $(C_{\text{out}}, C_{\text{in}}, k, k)$
偏置: $(C_{\text{out}},)$
输出: (C_{out}, H', W')

参数量公式

$$\#\text{params} = C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$$

算一算

$C_{\text{in}} = 3$, $C_{\text{out}} = 64$, $k = 3$:

$64 \times 3 \times 3 \times 3 + 64 = \mathbf{1792}$ 参数

设置	FC 层	Conv 层
输入	$32 \times 32 \times 3 = 3072$ 维	$3 \times 32 \times 32$
输出	1024 维	64 通道 $\times 32 \times 32$
参数量	$3072 \times 1024 \approx 314$ 万	$64 \times 3 \times 3 \times 3 \approx 1.7$ 千
比例	**FC 是 Conv 的 1800 倍**	

一个 CNN 的 “特征金字塔” 思想

- 浅层卷积：通道少、空间大 $\rightarrow$ 提取边缘、颜色等低级特征
- 深层卷积：通道多、空间小 $\rightarrow$ 提取语义、对象等高级特征
- 整体趋势：**空间尺寸 $\downarrow$，通道数 $\uparrow$**

为什么需要池化

- **降维**：减小特征图尺寸 → 减少计算
- **扩大感受野**：让深层神经元“看”到更大区域
- **增强鲁棒性**：对小幅平移、旋转不敏感
- **无参数**：不增加模型复杂度

两种主流池化

Max Pooling（最大池化）：取窗口内**最大值**
Average Pooling（平均池化）：取窗口内**平均值**

常见配置

2×2 窗口，stride=2 → **特征图尺寸减半**（高宽各减半）

4×4 输入，2×2 Max Pooling

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 0 & 8 & 7 & 9 \\ 4 & 2 & 5 & 3 \end{pmatrix}$$

窗口大小 2×2，stride=2

计算

左上窗口: $\max(1, 3, 5, 6) = 6$
右上窗口: $\max(2, 4, 1, 2) = 4$
左下窗口: $\max(0, 8, 4, 2) = 8$
右下窗口: $\max(7, 9, 5, 3) = 9$

结果

$$\mathbf{Y} = \begin{pmatrix} 6 & 4 \\ 8 & 9 \end{pmatrix}$$

尺寸从 $4 \times 4 \to 2 \times 2$

Max vs Avg 怎么选?

- **Max**：保留最强响应，更适合检测“是否存在某特征”。**现代 CNN 几乎都用 Max。**
- **Avg**：保留整体趋势，仅在少数场景使用（如全局池化）

ResNet 之后的趋势

现代 CNN（ResNet, EfficientNet, ConvNeXt）中，**中间池化层越来越少**，主要用 stride=2 的卷积来下采样。

为什么

- Stride 卷积可以**学习**怎样下采样，而池化是固定规则
- 池化 “无参数” 看似优点，实际限制了表达能力
- BatchNorm + 残差连接已经能很好控制信号

但池化没死

- **全局平均池化（GAP）**：取代分类前的全连接层（NiN/GoogLeNet/ResNet 都用）
- LeNet/经典网络：仍然广泛使用，作为入门概念也很重要

定义

某一层的某个神经元，在**原始输入图像**上能“看到”的区域大小。

直觉

- 第一层 3×3 卷积：每个神经元看到原图 3×3 区域
- 第二层 3×3 卷积：每个神经元看到第一层 3×3，对应原图 5×5
- 第三层 3×3 卷积：对应原图 7×7

感受野递推公式（无 stride）

$$R_l = R_{l-1} + (k_l - 1)$$

R_l：第 l 层感受野；k_l：该层卷积核大小

为什么要让感受野变大

要分类一只猫，最深层神经元应该能“看到”整只猫，否则它无法做出正确判断。

- 太小：只能看到局部 → 不能识别整体
- 太大：浪费计算
- 理想：覆盖目标对象大小

扩大感受野的三种方法

1. **堆叠更多卷积层**
2. **用 stride 下采样**（stride=2 让后续层感受野翻倍）
3. **使用更大的卷积核**（不推荐）

为什么不用大核？7×7 vs 三个 3×3

	7×7	$3 \times 3 \times 3$
感受野	7×7	7×7
参数	49	27
非线性层	1	3

结论（VGG 的核心洞察）

堆叠小核比单个大核：参数更少、非线性更多、表达力更强。

这是从 AlexNet 到 VGG 的关键设计变化。

背景

- Yann LeCun 1998 年提出
- 用于美国邮政手写数字识别
- 商用 30 年
- 奠定了 **Conv-Pool-FC** 范式

为什么先讲 LeNet

- 结构最简单
- 包含所有 CNN 核心元素
- MNIST 刚好能跑
- 是一切现代架构的 “祖先”

原版 LeNet-5 架构

1. Conv: 5×5, 6 通道 + Sigmoid
2. AvgPool: 2×2
3. Conv: 5×5, 16 通道 + Sigmoid
4. AvgPool: 2×2
5. FC: 120 + Sigmoid
6. FC: 84 + Sigmoid
7. FC: 10（输出）

现代版改动

- Sigmoid $\rightarrow$ ReLU
- AvgPool $\rightarrow$ MaxPool
- 加 BN（可选）

层	操作	输出形状	参数量
输入	—	$1\times28\times28$	0
Conv1	5×5, 6 通道, $p=2$	$6\times28\times28$	$6\times1\times5^2+6=156$
Pool1	MaxPool 2×2, $s=2$	$6\times14\times14$	0
Conv2	5×5, 16 通道	$16\times10\times10$	$16\times6\times5^2+16=2416$
Pool2	MaxPool 2×2, $s=2$	$16\times5\times5$	0
Flatten	展平	400	0
FC1	全连接	120	$400\times120+120=48120$
FC2	全连接	84	$120\times84+84=10164$
FC3	全连接（输出）	10	$84\times10+10=850$
总计			~ 6.2 万

重要观察

$\sim 95\%$ **参数在全连接层**。卷积层只有 ~ 2600 参数，却承担主要特征提取。

```python
import torch.nn as nn
class LeNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2),   # 1@28x28 -> 6@28x28
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),                   # 6@28x28 -> 6@14x14
            nn.Conv2d(6, 16, kernel_size=5),             # 6@14x14 -> 16@10x10
            nn.ReLU(),
            nn.MaxPool2d(2, stride=2),                   # 16@10x10 -> 16@5x5
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120), nn.ReLU(),
            nn.Linear(120, 84),     nn.ReLU(),
            nn.Linear(84, num_classes),
        )
    def forward(self, x):
        return self.classifier(self.features(x))
```

阅读技巧

注释里的形状对应上一页的尺寸演算表。**读 CNN 代码的核心：时刻跟踪 Tensor 形状。**

```python
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 数据准备
transform = transforms.Compose([
    transforms.ToTensor(),                  # PIL -> Tensor (1, 28, 28)
    transforms.Normalize((0.1307,), (0.3081,)),  # MNIST均值/方差
])
train_set = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_set  = datasets.MNIST('./data', train=False, transform=transform)
train_loader = DataLoader(train_set, batch_size=128, shuffle=True)
test_loader  = DataLoader(test_set,  batch_size=256)

# 模型、损失、优化器（第4-5讲学过的"标准配置"）
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = LeNet().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

这里的训练循环和第 5 讲完全一样

唯一改动：把第 5 讲的 MLP 换成 LeNet。**MNIST 上 LeNet 几个 epoch 就能达到 99%+ 精度。**

```python
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, total_correct, total_n = 0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss = criterion(logits, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * x.size(0)
        total_correct += (logits.argmax(1) == y).sum().item()
        total_n += x.size(0)
    return total_loss / total_n, total_correct / total_n

for epoch in range(5):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
    print(f"Epoch {epoch+1}: loss={train_loss:.4f}, acc={train_acc:.4f}")
```

常见输出（参考）

```
Epoch 1: loss=0.2841, acc=0.9152
Epoch 5: loss=0.0312, acc=0.9905
```

```python
import matplotlib.pyplot as plt

# 取出第一个卷积层的权重
conv1_weights = model.features[0].weight.data.cpu()  # (6, 1, 5, 5)
print(conv1_weights.shape)  # torch.Size([6, 1, 5, 5])

# 可视化6个5x5的卷积核
fig, axes = plt.subplots(1, 6, figsize=(12, 2))
for i, ax in enumerate(axes):
    ax.imshow(conv1_weights[i, 0], cmap='gray')
    ax.axis('off')
    ax.set_title(f'Filter {i}')
plt.tight_layout()
plt.show()
```

你会看到什么

- 几个核学到了**水平边缘检测器**
- 几个核学到了**垂直边缘检测器**
- 几个核学到了**斑点/角点检测器**
- **这是网络自动学到的，不是人工设计的！**

Bug 1: 形状不匹配（最常见）

`RuntimeError: shape [400] is invalid for input of size 256`

原因：FC 层输入维度算错。**排查**：在 `forward` 中加 `print(x.shape)` 逐层打印。

Bug 2: 输入通道数错

`Given groups=1, weight of size [6,1,5,5], expected input[B,3,28,28]`

原因：MNIST 是灰度图（1 通道），代码却传了 RGB 图（3 通道）。

Bug 3: 训练损失不下降

排查清单：学习率（10^{-3} 常用）/ 是否调用 `model.train()` / 数据是否归一化 / 标签是否 `long` 类型。

两个先验，两个新算子

- **局部性** + **平移不变性** → 卷积层（局部连接 + 参数共享）
- **聚合** + **鲁棒性** → 池化层

关键公式

- 卷积输出尺寸：$\text{out} = \lfloor (n + 2p - k)/s \rfloor + 1$
- Conv 参数量：$C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$
- 感受野递推：$R_l = R_{l-1} + (k_l - 1)$（无 stride）

经典结构

Conv → **ReLU** → **Pool** 反复堆叠 + 末尾全连接 = LeNet 范式
这就是 1998-2012 年间所有 CNN 的基本骨架。

下一讲（第 7 讲）：经典 CNN 架构与设计思想

- **AlexNet (2012)**：深度学习浪潮的导火索
- **VGG (2014)**：堆叠小核的极致
- **GoogLeNet (2014)**：Inception 模块与多尺度
- **ResNet (2015)**：残差连接，深度突破 1000 层
- **设计思想总结**：从手工设计到模块化

思考题（下次课讨论）

1. 为什么 LeNet 在 ImageNet 上不工作？
2. 把 LeNet 的卷积核从 5×5 改成 3×3，效果会怎样？
3. 如果只有 100 张训练图，CNN 还能用吗？

动手任务（建议在第 5 讲的 PyTorch 环境中完成）

1. **尺寸演算**：手算 LeNet 在 Fashion-MNIST（$1 \times 28 \times 28$）上每层的输出形状
2. **代码实现**：参考本讲代码，搭建并训练 LeNet 分类 Fashion-MNIST，目标精度 $\geq 90\%$
3. **对比实验**：把 LeNet 的两个 5×5 卷积换成两个 3×3 卷积，比较参数量和精度
4. **可视化**：将训练好的第一个卷积层的 6 个核可视化，描述你看到的模式
5. **选做**：在 LeNet 基础上加 BatchNorm（参考第 4 讲），观察训练曲线变化

谢谢！

问题与讨论

下次课：经典 CNN 架构演进

孟翰　人工智能学院